arXiv:2406.06282v3 [cs.LG] 12 Dec 2024

PowerInfer-2: Fast Large Language Model Inference on a Smartphone

Zhenliang Xue*, Yixin Song*, Zeyu Mi✉, Xinrui Zheng, Yubin Xia, and Haibo Chen

Institute of Parallel and Distributed Systems (IPADS), Shanghai Jiao Tong University

Abstract

Large language models (LLMs) on smartphones enable real-time AI assistance and privacy-preserving, offline operation. However, resource constraints of smartphones limit current deployments to small language models (SLMs), significantly compromising their capabilities. This paper introduces *PowerInfer-2*, a smartphone-based framework that enables fast inference for LLMs exceeding the memory capacity. The key insight is decomposing matrix operations into neuron clusters as the basic processing unit, which enables flexible scheduling and efficient I/O-computation pipelining. PowerInfer-2 leverages this neuron-cluster-based design in both computation and storage. For computation, neuron clusters with dense activations are processed on NPU, while sparse clusters use CPU. The storage engine provides a fine-grained pipeline mechanism that coordinates cluster-level computation and I/O operations, enhanced by a segmented neuron cache to reduce I/O activities. PowerInfer-2 achieves up to a 27.8× speed increase compared to state-of-the-art frameworks. PowerInfer-2 is the first system to serve a 47B LLM on a smartphone, achieving 11.68 tokens/s. Notably, these performance improvements preserve model quality with negligible accuracy degradation.

1 Introduction

Large Language Models (LLMs) have revolutionized our daily lives and work patterns through their remarkable ability to process and generate human-like text. While state-of-the-art LLMs such as GPT-4 [45] and Claude-3.5 [9] operate on powerful data center GPUs (e.g., NVIDIA H100 [42] and GB200 [43]), there is an increasingly growing interest in deploying LLMs directly on ubiquitous smartphones. This trend is evidenced by major tech companies' recent initiatives in mobile LLM integration: Apple has launched Apple Intelligence [10], Google has introduced Gemini Nano for Pixel devices [22], and smartphone manufacturers including Samsung, Huawei, and OPPO have incorporated various LLM solutions into their smartphones [2,4,5]. The surge of interest in mobile LLM deployment stems from three key advantages: it enables instantaneous AI assistance across system functions by eliminating network latency, enhances user privacy through local data processing, and ensures reliable performance regardless of network conditions.

However, mobile LLM deployment faces a critical challenge: due to smartphones' limited computing and memory resources, only small language models (SLMs) can be deployed, significantly compromising model quality and capabilities. For example, Google's Gemini Nano [22], specifically designed for smartphones, contains only 3.25B parameters and requires less than 2GB of memory, which is substantially smaller than models deployed on high-end PCs and servers (typically ranging from tens to hundreds of billions of parameters). This model size constraint severely limits the model's capabilities, as larger models consistently demonstrate superior performance according to the "scaling law" [30].

To address these limitations, recent research focuses on system-level optimizations that leverage sparse activation patterns, with solutions like PowerInfer [51] and LLMFlash [7] demonstrating promising results in high-end PC environments. However, these PC-centric solutions suffer from severe execution speed degradation on smartphones (i.e., approximately 70% slower decoding speed for 7B models) due to two key hardware gaps between smartphones and high-end PCs:

- *The Sparse Computing Gap:* Smartphones' mobile NPUs, while offering superior performance for dense computations compared to mobile CPUs, lack dedicated hardware support for sparse operations and struggle with unstructured sparse computations. In contrast, high-end PCs' discrete GPUs efficiently handle both dense and sparse matrix operations compared to CPUs. This hardware limitation significantly impacts the effectiveness of PowerInfer and LLMFlash's sparse computation approaches on smartphones, as sparse computations are forced to run on CPUs. Such CPU-only execution not only underutilizes the NPU's computational capabilities but also fails to fully exploit the high memory bandwidth available in smartphones' unified memory architecture (UMA).
- *The Storage Performance Gap:* The Universal Flash Storage (UFS) in smartphones faces severe performance limitations compared to PC's NVMe SSDs, achieving

*Co-first authors.

✉Corresponding author: Zeyu Mi (yzmizeyu@sjtu.edu.cn).

merely half the bandwidth for sequential reads (4GB/s vs. 7.5GB/s), while for random reads, this performance gap widens dramatically to 12× (100K IOPS vs. 1,200K IOPS). Consequently, despite its specialized I/O optimizations, LLMFlash still cannot overcome these fundamental storage bandwidth limitations, resulting in frequent computational unit idling during I/O operations.

This paper presents *PowerInfer-2*, an LLM-serving framework that efficiently runs large language models on smartphones, achieving high inference speeds even for models exceeding available device memory. Unlike existing solutions primarily targeting high-end PC environments, PowerInfer-2's core innovation lies in its fine-grained neuron cluster abstraction for mobile LLM inference. A neuron cluster represents a collection of neurons exhibiting the same activation pattern, and its neuron number is dynamically determined at runtime based on the target computing units. This fine-grained abstraction enables both optimal utilization of heterogeneous computing resources and efficient coordination between computation and I/O operations.

Based on the flexible neuron cluster abstraction, PowerInfer-2 provides two key principles that redesign LLM inference to address smartphone-specific hardware constraints (e.g., NPU-CPU heterogeneity, UFS limitations). First, given smartphones' limited computing capabilities compared to PCs, it is crucial to maximize performance by intelligently distributing tasks between NPU and CPU based on their respective computational strengths. Specifically, PowerInfer-2's *Sparsity-Aware Adaptation* simultaneously assigns neuron clusters of different sizes to NPUs for efficient dense matrix operations and CPUs for flexible sparse computations. This parallel execution not only maximizes computational throughput but also optimizes memory bandwidth utilization. Moreover, PowerInfer-2 redistributes the ratio of neuron clusters between NPU and CPU to dynamically adapt to changes in sparsity patterns across different batch sizes, which is a critical optimization overlooked by previous systems.

The second principle, *I/O-Aware Orchestration*, addresses a fundamental challenge in mobile LLM inference: the mismatch between traditional matrix-based computation granularity and mobile storage performance. Conventional matrix-level operations often leave computing units idle while waiting for slow UFS I/O, severely underutilizing available computing resources. To solve this, PowerInfer-2 leverages its neuron cluster-based computation model to efficiently pipeline matrix operations, which brings two key benefits: First, it enables efficient overlap between computation and I/O, effectively hiding storage access latency. Second, it allows the system to choose specialized I/O access patterns for different neuron clusters, improving storage bandwidth utilization during inference.

We implemented PowerInfer-2 by extending PowerInfer [51], adding 12K lines of code. PowerInfer-2 has been evaluated on two smartphones: OnePlus 12 and Ace 2, all featuring heterogeneous Qualcomm XPUs with 24GB and 16GB DRAM. Our system fundamentally breaks through the SLM constraints on smartphones, enabling deployment of large language models that were previously impossible to run on mobile devices. Specifically, PowerInfer-2 supports various LLM architectures and model sizes (from 7B to 47B). Our evaluation shows three key improvements: (1) Performance: 24.6× (up to 27.8×) over llama.cpp and 3.84× average speedup (up to 4.63×) over LLMFlash on OnePlus 12; (2) Model size: First system to run large models like TurboSparse-Mixtral-47B on mobile, at 11.68 tokens/s; (3) Memory: 40% less memory usage for 7B models while matching llama.cpp and MLC-LLM speed. Notably, these improvements maintain model quality with minimal accuracy impact.

2 Background and Motivation

2.1 LLM Inference Background

LLM inference operates in two distinct phases: prefill and decoding. In the prefill phase, the model processes the entire user prompt in parallel, producing the initial token. Subsequently, in the decoding phase, the model generates text through an autoregressive process, where each newly generated token becomes the input for producing the next one, continuing until either the desired output length is reached or an end-of-sequence (EOS) token is encountered.

Contemporary LLMs predominantly utilize a decoder-only transformer architecture, consisting of multiple transformer layers. Each layer incorporates an attention mechanism and a Feed-Forward Network (FFN). With the adoption of Group Query Attention [47] in recent models, the FFN component has become the dominant factor in model size, accounting for approximately 80% of total parameters in models like Llama3-8B, Qwen2-7B, and Mistral-7B. This dominance makes FFN design choices crucial for inference performance. Notably, ReLU-family activation functions [6, 48, 62] have gained prominence in this context: they not only achieve comparable model performance to non-ReLU alternatives, but also introduce significant sparsity characteristics [34, 39, 61], as many neurons naturally become inactive due to their negligible contribution to the final output.

2.2 LLM Inference with Dynamic Sparsity

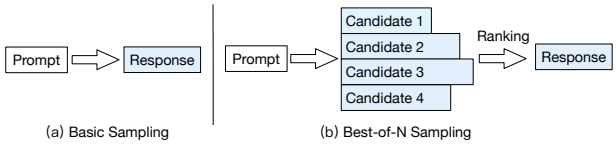


Figure 1: Comparison of sampling strategies used in LLM inference. (a) Basic sampling generates a single response directly from the prompt. (b) Best-of-N sampling generates multiple candidate response sequences for one prompt, and selects the final response through a ranking process.

Prior works [7, 37, 50, 51, 62] have demonstrated that FFN

block activations can be predicted before computation. PowerInfer [51], for instance, employs a *static* approach, distributing hot and cold neurons between GPU and CPU based on predicted activation patterns. However, this approach overlooks an important aspect of modern LLM deployments: the dynamic sparsity patterns introduced by advanced decoding strategies. While traditional methods generate tokens sequentially using basic sampling (Fig.1-a), recent innovations have introduced advanced decoding methods that generate and evaluate multiple candidate sequences in parallel. As illustrated in Fig.1-b, techniques such as Best-of-N (BoN) sampling [35, 49] generate multiple candidates and select the best response. Similar parallel generation strategies are also employed in Monte Carlo Tree Search (MCTS) [60] and map-reduce paradigms [64].

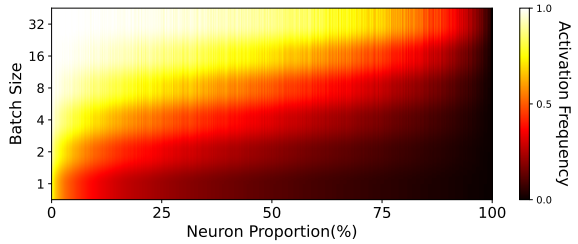


Figure 2: Neuron activation patterns in layer 10 of Bamboo-7B under different batch sizes. The X-axis represents the proportion of neurons (sorted by activation frequency), and the Y-axis shows different batch sizes. Darker red indicates lower activation frequency.

As shown in Fig.2, parallel token generation leads to dynamic sparsity patterns in neuron activations[1] during inference. Our analysis uncovers two key insights that current LLM frameworks fail to exploit: First, while large batch sizes exhibit consistent "hot spots" of activated neurons, single-batch processing demonstrates higher sparsity with more dispersed, irregular activation patterns. Second, during decoding, the effective batch size exhibits dynamic fluctuations due to varying sequence termination points. The proportion of highly-activated neurons (depicted in white in Fig.2) demonstrates a dramatic correlation with batch size, escalating from less than 1% at batch size 1 to approximately 75% at batch size 32. These two insights not only indicate that an efficient inference system should exploit sparsity patterns between hot and cold neurons, but also needs to dynamically adapt to their shifting distributions.

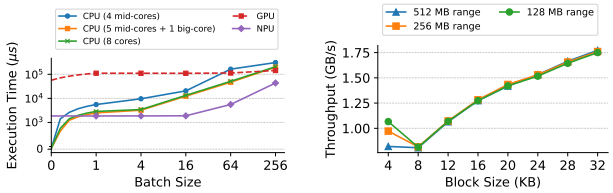
2.3 Mobile Hardware Characteristics

Mobile devices present fundamentally different hardware characteristics from PCs, manifesting in two key aspects that directly impact LLM inference: (1) heterogeneous computing capabilities with distinct sparse computation characteristics, and (2) distinct storage architecture with unique I/O characteristics.

[1] A neuron is considered activated when triggered by at least one token in the batch.

2.3.1 Computing Unit Characteristics

Unlike high-end PC environments where discrete GPUs consistently dominate LLM computations, smartphones feature multiple computing units (including CPU, GPU, NPU, etc.) with distinct performance characteristics, particularly in handling sparse computations. Taking OnePlus 12 (equipped with a Qualcomm Snapdragon 8 Gen 3 processor) as a case study, we analyze how each processor type handles dense versus sparse computations in LLM inference tasks.



(a) **(b)**

Figure 3: (a) Comparison of execution times for matrix-vector multiplication operations with varying computational loads across CPU, GPU, and NPU platforms. The test involves multiplying a 14336×4096 matrix with vectors under different batch sizes. The X-axis represents the batch size while the Y-axis shows execution time. (b) Random read throughput performance for 4KB operations across different block sizes and data ranges. The X-axis depicts block size, while the Y-axis represents throughput.

CPU's Advantage in Sparse Computing. Mobile CPUs utilize big.LITTLE technology [11], combining high-performance "big" cores with high-efficiency "little" cores. The Snapdragon 8 Gen 3 features a 1+5+2 configuration: one big-core for peak performance, five mid-cores for balanced performance, and two little-cores for energy efficiency. Our experiments reveal that mobile CPUs, particularly the big and middle cores, excel at handling sparse matrix-vector computations. When handling sparse computations (matrices with batch size less than 1), six CPU cores demonstrate faster performance compared to NPUs and GPUs, as illustrated in Fig.3-a.

NPU's Dense Computing Strength. The Neural Processing Unit (NPU), while efficient for dense computations, shows limitations in sparse operations. As shown in Fig.3-a, NPUs excel at large batch processing, performing significantly faster than CPUs and GPUs at high batch sizes. Therefore, for a 7B INT4 model, NPU achieves a throughput of 770 tokens/s for the prefill phase[2], significantly outperforming GPU's 37 tokens/s and CPU's 8 token/s. However, our implementation of sparse computation primitives on Qualcomm's NPU actually performs slower than CPU implementations, which is stark contrast to PC platforms where GPUs excel at both dense and

[2] Qualcomm's NPUs typically employ coarse-grained per-channel quantization that leads to accuracy degradation. This issue can be fixed by adopting a mixed-precision quantization approach inspired by AWQ [15, 36], which preserves outlier weights while applying INT4 quantization to the remaining components, effectively maintaining model accuracy.

sparse computations.

GPU's Limited Role. Mobile GPUs show consistent performance limitations in our tests. Fig.3-a demonstrates that for matrix-vector operations, GPU consistently performs slower than both NPU and CPU, with only about 50% of GPU kernel time spent on actual computation. Moreover, since GPUs are primarily responsible for rendering tasks, utilizing them for inference would significantly impact the device's rendering frame rate [32]. Therefore, due to these performance limitations and rendering resource contention, our private communications with mobile manufacturers reveal that they avoid using GPUs for LLM inference in production systems.

XPU and Memory Bandwidth Sharing. Smartphone's unified memory architecture (UMA) also differs significantly from PC's separate VRAM and storage design. Under this architecture, all processors share the same memory space, enabling efficient memory bandwidth sharing. Our measurements on OnePlus 12 running a 7B parameter model show that memory bandwidth varies significantly based on processor utilization. When performing matrix multiplication operations, CPU-only computations achieve 43.9GB/s bandwidth, while NPU-only computations reach 56GB/s. However, when coordinating both processors simultaneously, the aggregate memory bandwidth increases to 59.6GB/s. This suggests that mobile LLM inference frameworks should leverage heterogeneous processors simultaneously to maximize the utilization of shared memory bandwidth.

2.3.2 Storage I/O Characteristics

The second key challenge arises from the fundamental differences between mobile and PC storage architectures. Mobile devices primarily use Universal Flash Storage (UFS), which exhibits markedly different I/O characteristics compared to the NVMe SSDs found in PCs. Through our comprehensive analysis of UFS 4.0, we identified four distinctive performance characteristics that significantly impact LLM inference:

Block Size Impact. Read bandwidth varies significantly with block size. For sequential reads, bandwidth ranges from 450 MB/s with 4KB blocks to 4 GB/s with 512KB blocks. Random read performance shows similar variation, reaching 3.5 GB/s with 512KB blocks but dropping significantly with smaller blocks.

Data Range Sensitivity. Random read performance is highly sensitive to the scope of the read range. As shown in Fig.3-b, 4KB random reads within a 128MB range achieve 1 GB/s, while the same operations across a 512MB range drop below 850 MB/s. This characteristic is particularly pronounced for small block sizes, making data locality crucial for performance optimization.

CPU Core Dependencies. Storage performance is tightly coupled with the CPU core handling I/O operations. As shown in Table 1, using a big-core (3.3GHz) achieves 1 GB/s for 4KB random reads, while a little-core (2.2GHz) only reaches 760 MB/s. This correlation stems from the CPU's role in processing UFS driver operations, including interrupt handling and queue management.

Table 1: 4KB random read throughputs on 128MB data range with different core setups.

Core Setup	Throughput (MB/s)
big-core (3.3GHz)	1,076.10
mid-core (3GHz)	1,007.95
little-core (2.2GHz)	761.87

Limited Concurrency. Unlike NVMe SSDs with multiple command queues, UFS storage has a single command queue, limiting internal concurrency. Our tests show that using multiple cores for I/O operations can degrade performance by up to 40% due to command queue contention.

2.4 Limitations of Existing Solutions

These mobile hardware characteristics pose significant challenges for existing LLM inference frameworks. To quantitatively analyze the performance bottlenecks of existing solutions on mobile platforms, we implemented and evaluated both PowerInfer and LLMFlash on a OnePlus 12 smartphone running the Mistral 7B model. We set a small memory budget for the evaluation and offloaded 50% of FFN weights to flash storage. Since PowerInfer was originally designed without flash storage support, we extended its design to handle cases where model weights exceed available memory. Both systems use Asynchronous I/O (AIO) to access cold neurons stored in flash storage.

Table 2: Decoding speeds and FFN performance in of Mistral 7B on existing methods w/wo offloading.

Solutions	In Memory		Offloading 50% FFN Weight		
	Decoding Speed	Memory Bandwidth	Decoding Speed	I/O Overhead	CPU Utilization
PowerInfer	12.4 tok/s	41.1 GB/s	1.4 tok/s	81.9%	31%
LLMFlash	12.9 tok/s	43.9 GB/s	2.3 tok/s	76.7%	41%

Our experimental results demonstrate severe performance degradation when offloading 50% of FFN weights to flash storage. As shown in Table 2, PowerInfer experiences a 89% decoding speed reduction, while LLMFlash suffers a 82% decline. This significant performance gap can be attributed to two fundamental challenges in the mobile computing environment. First, existing solutions' computational strategies are heavily dependent on hardware-accelerated sparse operations. While this approach is effective on PC platforms with powerful GPUs, it becomes problematic on mobile devices where NPUs lack efficient sparse computation capabilities, ultimately forcing all sparse computations onto the CPU. Our measurements from Table 2 reveal that even with weights stored in memory, mobile memory bandwidth utilization reaches only approximately 43.9 GB/s due to CPU cores' limited computing capabilities, which is far below 59.6GB/s when both NPU and CPU are fully engaged. Second, the significantly lower I/O bandwidth of mobile UFS storage (compared to PC's NVMe SSDs) creates a major performance

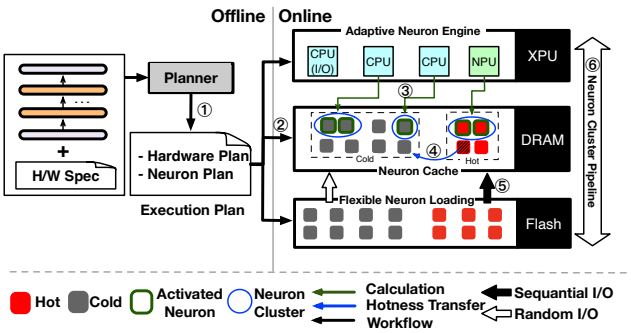


Figure 4: The architecture overview of PowerInfer-2.

bottleneck, with I/O overhead accounting for 82% and 77% of total latency in PowerInfer and LLMFlash respectively. The slow I/O operations force computing units to frequently idle while waiting for I/O operations, with CPU utilization dropping to 41% during weight loading phases.

3 PowerInfer-2 Overview

3.1 Neuron Cluster and Design Principles

This paper proposes neuron clusters as the basic unit for mobile LLM inference. A neuron cluster consists of a group of neurons from the same FFN layer that exhibit similar activation patterns. The cluster size is adaptively determined and categorized based on both hardware characteristics and activation frequencies: larger hot clusters comprising frequently activated neurons are allocated for NPU-accelerated dense computations, while smaller cold clusters containing rarely activated neurons are handled by CPU-driven sparse computations.

PowerInfer-2 builds upon this abstraction with two design principles. First, **Sparsity-Aware Adaptation** maximizes hardware utilization through intelligent workload distribution. During prefill phase, the system leverages NPU for efficient dense operations. During decoding phase, it dynamically allocates workload between CPU and NPU based on sparsity patterns and batch sizes. Second, **I/O-Aware Orchestration** optimizes UFS storage access through fine-grained pipelining and differentiated I/O strategies for various types of LLM weights.

3.2 Overall Architecture

Fig.4 presents the overall architecture of PowerInfer-2, which consists of an offline planning phase and an online inference phase. Due to the dynamic sparsity patterns and heterogeneous computing characteristics of mobile platforms, the offline phase needs to analyze both model characteristics and hardware capabilities to generate optimized execution plans. During online inference, PowerInfer-2 proposes four techniques and mechanisms (§4) to deliver efficient inference on mobile devices.

Offline Planning: The planner first conducts offline analysis by running the model to understand activation patterns across different batch sizes. It classifies neurons into hot clusters and cold clusters through a comprehensive process that considers both model characteristics and hardware capabilities. Based on this analysis, it generates an execution plan (Step ①) including a neuron plan that guides how to split neurons into clusters, and a hardware plan that guides how to configure the hardware resources for PowerInfer-2 (Step ②).

Online Inference: During inference, an adaptive neuron engine (Step ③) efficiently orchestrates workload distribution between NPU and CPU based on the neuron plan and dynamic sparsity patterns. The engine implements two specialized strategies: During the prefill phase, it uses an NPU-centric approach where the NPU handles dense matrix operations while a dedicated CPU core performs parallel weight loading. For the decoding phase, the engine adopts a CPU-NPU hybrid strategy that optimally distributes workload - assigning hot neuron clusters to the NPU while delegating cold clusters to the CPU. Similar to PowerInfer and LLMFlash, the CPU side features an online predictor to minimize computation overhead by selectively calculating only those neurons predicted to be activated. To accommodate dynamic sparsity patterns, the engine continuously monitors and adjusts neuron cluster allocation between NPU and CPU as batch sizes evolve over time (Step ④).

Prior to initiating inference computations, the computing engine retrieves neuron weights from a neuron cache specifically designed to exploit distinct access patterns of different LLM weights. For weights not present in the cache, PowerInfer-2 introduces an adaptive neuron loading mechanism that uses differentiated I/O strategies to maximize efficiency (Step ⑤). To mitigate the performance impact of random reads, PowerInfer-2 introduces a neuron cluster pipeline mechanism (Step ⑥). For instance, upon completing the processing of a neuron cluster from matrix A, CPU cores immediately transition to processing pre-cached neuron clusters from matrix B. This operation executes concurrently with ongoing I/O operations for matrix A's in-flash neuron clusters, enabling optimal parallelization between computation and I/O operations.

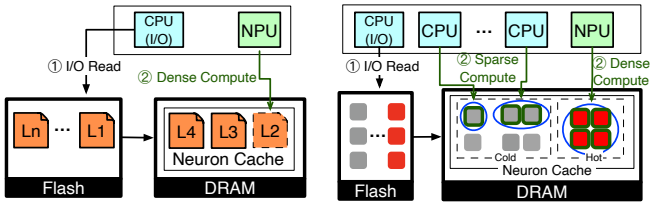
4 Detailed Design of Online Inference

4.1 Adaptive Neuron Engine

PowerInfer-2 introduces an adaptive neuron engine that proposes flexible computation strategies based on different inference stages and dynamic sparsity, fully leveraging the computational capabilities of mobile XPUs.

4.1.1 NPU-Centric Prefill

In the prefill phase, all prompt tokens are processed concurrently. Even though individual tokens exhibit high sparsity and activate distinct neurons, the overall sparsity decreases significantly when aggregating these activations across the batch. Consequently, PowerInfer-2 merges all neurons into a large cluster during the prefill stage rather than using pre-



(a) NPU-Centric Prefill (b) CPU-NPU Hybrid Decoding

Figure 5: Two computing workflows for prefill and decoding phases. (a) The prefill phase uses an NPU-centric workflow that leverages NPU for computation; (b) The decoding phase employs a CPU-NPU hybrid workflow for FFN computation where NPU handles dense computations for hot neurons while CPU cores process sparse computations for cold neurons, with their processing ratio automatically adjusting to match the dynamic sparsity patterns caused by varying batch sizes. The attention computation is handled entirely by NPU but not shown in the figure.

dictors to calculate activated neurons. Given NPU's superior performance in large batch matrix operations compared to CPU, PowerInfer-2 primarily leverages NPU for prefill computations. Although CPU cores do not participate in matrix calculations during prefill, the neuron engine leverage a CPU core to load all weights sequentially from flash into memory.

Fig.5-a shows how CPUs and NPU work together for prefill-phase inference. The NPU processes all LLM layers sequentially via its powerful dense matrix operations. The NPU shares limited memory with the CPU, so the CPU preloads matrix weights into this shared memory before NPU computation. PowerInfer-2 uses a big-core to asynchronously preload weights for the next layer into the neuron cache. Since the prefill stage involves dense matrix calculations, sequential reads are used to load large data blocks, maximizing UFS's I/O bandwidth.

4.1.2 Hybrid CPU-NPU Decoding

During decoding, PowerInfer-2 adopts distinct strategies for attention and FFN computations (Fig.5-b). For attention blocks, which are inherently dense but relatively small, PowerInfer-2 partitions and distributes individual attention heads between CPU and NPU, with each XPU processing its assigned heads in parallel.

For FFN blocks that dominate computation, PowerInfer-2 adopts a hybrid approach where NPU handles dense hot neurons while CPU processes sparse cold neurons. The neuron engine splits an FFN matrix along the neuron dimension (rows), with the split ratio dynamically adjusting based on batch size. For example, when batch size is 1, it divides a typical FFN matrix (14336×4096) into two equal sub-matrices (7168×4096) for NPU and CPU respectively. As batch size increases, the NPU portion grows larger to handle more dense computations. The NPU performs dense matrix multiplication on its portion, while CPU cores process cold neurons with a predictor-based approach [51]. The CPU further divides its workload into smaller chunks (e.g., 512×4096) for parallel processing, utilizing ARM Neon extensions for vectorized computation. After both XPUs complete their computations, the results are merged by the CPU side to form the final output.

4.1.3 Dynamic CPU-NPU Adjustment

To handle dynamic sparsity patterns efficiently, the adaptive neuron engine dynamically adjusts the CPU-NPU computation ratio based on runtime batch sizes. CPU cores monitor batch size changes by tracking the completion and creation of decoding sequences. For large batch sizes with multiple concurrent sequences, sparsity is low due to high neuron activation overlap. In this case, NPU handles most computations (about 70%) while CPU processes the remaining sparse part (about 30%). As sequences complete over time and batch size decreases, sparsity increases, shifting more workload to CPU cores (up to 50% for batch size 1) while NPU focuses on remaining hot neurons.

Given NPU's static graph execution model, adjusting its computational load requires loading new computation graphs. During the offline phase, PowerInfer-2 prepares multiple NPU computation graphs, each optimized for a specific batch size and corresponding hot neuron ratio. These graphs differ in their operator shapes based on the number of participating neurons. When the engine detects the need for NPU load adjustment during runtime, it initiates an asynchronous graph loading process while NPU is computing the attention block. The new graph, typically only 10KB in size, is loaded into NPU memory and becomes active immediately after attention computation completes. This asynchronous loading mechanism completely overlaps with NPU computation, ensuring seamless transitions between different computational configurations without introducing additional overhead.

4.2 In-Memory Neuron Cache

Efficient in-memory caching is crucial for optimizing inference performance by minimizing the frequency of I/O operations. While prior work like LLMFlash [7] proposes bundling co-activated neurons to reduce I/O operations, this approach faces two key limitations on mobile devices: First, it overlooks the skewed activation patterns where a small set of hot neurons dominates the connections, leading to redundant loading of these frequently activated neurons across different bundles. Second, our analysis shows that after excluding hot neurons, the co-activation probability among remaining neurons drops below 20%, making the bundling strategy ineffective for I/O reduction.

To overcome these limitations, PowerInfer-2 implements a temperature-based caching strategy with two objectives: eliminating redundant loading by explicitly distinguishing hot and cold neurons, and optimizing cache efficiency for cold neurons through individual management rather than bundling, given their low co-activation probability. Specifically, the cache consists of three regions: (1) a fixed region for attention weights and KV cache that are preloaded and retained; (2) a

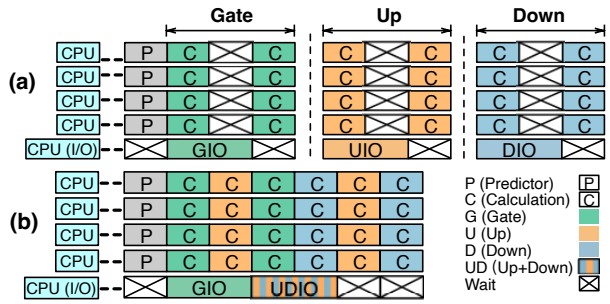


Figure 6: Two types of pipelines that combine matrix-vector multiplications and I/O operations on five cores (4 calculation cores and 1 I/O cores). Assuming that there are 8 neuron cluster per matrix, 4 of them are in memory and the other 4 are in flash before the computation starts. (a) The matrix-level pipeline separates the pipeline into isolated matrix units; (b) The neuron-cluster-level pipeline in PowerInfer-2 breaks the matrix barrier and mixs their calculation and I/O operations in the neuron cluster granularity.

hot region for NPU dense computations; and (3) a cold region for CPU sparse computations.

The hot and cold regions employ different organizational strategies. The hot region groups neurons into dense matrices, combining Gate, Up, and Down sub-matrices mentioned in §4.1.2 into a unified hot neuron cluster. In contrast, the cold region manages neurons individually. Both regions use LRU-based eviction, but at different granularities: cluster-level for the hot region and neuron-level for the cold region. Evicted weights from both regions are simply discarded from memory without being written back to storage.

When batch sizes change, PowerInfer-2 dynamically adjusts the hot-cold region ratio. For larger batches, the hot region expands to support increased NPU computations, while the cold region shrinks by evicting the least active neurons via LRU policy. Conversely, for smaller batches, the hot region also discards the least active neuron clusters to make room for the cold region.

4.3 Neuron-Cluster-Level Pipeline

Equipped with the neuron cache, I/O operations for uncached weights remain inevitable. While attention blocks and hot neurons benefit from efficient sequential reads, cold neurons require random I/O operations due to their sparse activation patterns. This results in significantly reduced bandwidth due to UFS's poor random read performance, creating a major bottleneck that impacts systems like PowerInfer and LLMFlash on mobile devices (§2.4).

Therefore, PowerInfer-2 should hide I/O latency by overlapping computation with I/O operations. A straightforward approach is matrix-level overlapping: processing cached neurons while simultaneously loading uncached neurons from storage. However, this approach still has limitations. The system must wait for all neurons within a matrix to be available before proceeding to the next matrix, especially when some neurons require lengthy I/O operations. For example, as shown in Fig.6-a, with a matrix containing 8 neuron clusters (4 in memory, 4 in storage), CPU cores may still experience idle periods waiting for I/O completion, despite partial overlapping of computation and I/O.

To minimize I/O latency, PowerInfer-2 introduces a neuron-cluster-level pipeline that tightly integrates computation and I/O activities, following the **I/O-Aware Orchestration** principle. Concretely, PowerInfer-2 breaks down the barriers between matrix computations; as soon as one neuron cluster finishes computing, it immediately starts the computation of a neuron cluster in the next matrix that are in memory. This mechanism overlaps the I/O operations within neuron cluster computations from multiple matrices, effectively reducing waiting bubbles, as illustrated in Fig.6-b.

PowerInfer-2 divides the execution process of a neuron cluster into 5 sequential stages, which are: determining whether the rows/columns of the Gate, Up, and Down matrices is activated through the predictor (Pred), reading the weights of the rows of the Gate matrix from storage (GIO), calculating the product of the rows of the Gate matrix and the input vector (GC), reading the rows/columns of the Up and Down matrices from storage (UDIO), and calculating the product of the rows/columns of the Up and Down matrices with the input vector respectively (UDC). PowerInfer-2 creates multiple computing threads and one I/O thread to handle the computations and I/O operations for these 5 stages, respectively. The specific number of these threads and the cores on which they will execute are determined by the offline planner.

4.4 Flexible Neuron Loading

PowerInfer-2's I/O-aware orchestration employs distinct I/O strategies for different weight types. For attention weights that are used densely during prefill, PowerInfer-2 loads them entirely into memory along with other parameters using sequential reads, and retains them in the attention region of the cache throughout inference. For FFN hot neurons, PowerInfer-2 also utilizes sequential reads but employs asynchronous preloading during attention computation to ensure immediate availability for FFN operations. For cold neurons that exhibit sparse activation patterns, PowerInfer-2 adopts an on-demand loading approach with small random reads.

To further optimize cold neuron loading efficiency, PowerInfer-2 bundles correlated neurons across Gate, Up, and Down matrices. While neurons within a single FFN matrix rarely co-activate after removing hot neurons, corresponding neurons at the i-th position across these matrices show high correlation, with an 80% co-activation probability. Thus, PowerInfer-2 organizes storage by neuron position rather than matrix structure, storing weights of corresponding neurons from all three matrices as a single unit.

PowerInfer-2 further introduces distinct I/O loading strategies for different models, considering their quantization methods and the inherent characteristics of UFS I/O. For unquantized models like Mistral-7B-FP16, where each neuron occu-

pies 8KB, PowerInfer-2 performs single large random reads of 24KB to load the complete Gate-Up-Down bundle, maximizing I/O bandwidth efficiency. For 4-bit quantized models, each neuron bundle (Gate-Up-Down) requires 7.5KB (2KB int4 weights + 0.5KB FP16 scales per matrix). PowerInfer-2 aligns the bundle size to 8KB for storage efficiency, but strategically divides loading into two separate 4KB operations, as empirical measurements demonstrate superior bandwidth utilization compared to a single 8KB random read (§2.3.2).

Moreover, while these bundles show 80% co-activation rate, there is still a 20% chance that bundled neurons do not activate together. Loading the entire bundle at once could waste bandwidth. For 4-bit quantized models, PowerInfer-2 adopts a two-phase loading strategy where it first loads the Gate matrix weights (4KB) after predictor confirms activation, and only loads the Up/Down weights (4KB) if the Gate neuron output is non-zero. This approach minimizes unnecessary I/O operations while maintaining performance.

5 Execution Plan Generation

To enable efficient LLM inference across diverse mobile platforms, PowerInfer-2 employs a targeted offline planning phase that generates device-specific execution plans. This planning phase optimizes for different runtime scenarios and dynamic batch sizes, automatically adapting neuron assignments between NPU and CPU based on each device's unique hardware capabilities.

Neuron Classification: The planner's primary task is to analyze and classify neurons based on their activation patterns across different batch sizes and hardware I/O characteristics. This classification process involves: (1) running the model on a diverse dataset (10M+ tokens from Wikipedia and RefinedWeb), (2) tracking activation frequencies for each neuron under various batch sizes, (3) analyzing hardware-specific I/O patterns and computation speeds, and (4) categorizing neurons into hot clusters and cold clusters.

The classification considers two key factors: First, the inherent activation patterns of neurons under different batch sizes. Second, the hardware I/O capabilities of the target device. Since hot neurons are asynchronously prefetched during the previous attention block's computation, PowerInfer-2 carefully balances the number of hot neurons based on the available I/O bandwidth and attention block computation time. This ensures that asynchronous I/O operations are completely hidden within the attention computation process.

For example, during the prefill phase, dense activation patterns dominate, leading to NPU processing assignments. For the decoding phase, the planner performs detailed analysis across batch sizes (1-4). For larger batches (e.g., 3-4), more neurons are classified as hot clusters due to denser activation patterns. For smaller batches (1-2), more neurons are classified as cold clusters due to sparse activation patterns. These assignments are further refined based on the device's specific I/O characteristics to optimize the overlap between computation and data movement.

Table 3: Hardware specifications of smartphones we used in the evaluation. "DRAM" is the physical memory size. "Available" is the maximum memory size that can be occupied by an application.

Device Name	DRAM / Available	Storage	SoC
OnePlus 12	24GB / 19GB	UFS 4.0	Snapdragon 8 Gen 3
OnePlus Ace 2	16GB / 11GB	UFS 3.1	Snapdragon 8+ Gen 1

Batch-Adaptive Planning: Based on the neuron classification results, the planner prepares a series of execution strategies optimized for different batch sizes. These strategies include pre-generated static NPU computation graphs for common batch sizes, and I/O patterns tailored to each execution mode. This pre-planning enables the runtime system to quickly adapt its execution strategy when batch size changes occur, avoiding the overhead of real-time analysis while maintaining optimal performance across different inference phases.

Hardware-Aware Optimization: The planner also considers hardware capabilities when generating these batch-adaptive strategies. It profiles the target device's NPU performance characteristics for different matrix sizes, CPU capabilities for sparse computation, memory bandwidth under various access patterns, and storage I/O performance for different access sizes. These hardware insights enable PowerInfer-2 to automatically adapt its execution strategy to different mobile platforms, ensuring optimal performance across diverse devices.

6 Implementation

PowerInfer-2 is developed on top of PowerInfer [51], a state-of-the-art serving framework designed for sparsely-activated LLMs, by integrating an additional 12K lines of C++ code into PowerInfer [51]. Since PowerInfer-2 depends on privileged system APIs (e.g., `mlock` that locks pages in memory) that need the root permission, we built it on the Android [8] platform. Even though there is no need to alter the system kernel, a rooted Android system still provides us with considerable flexibility in developing and debugging our system. Furthermore, PowerInfer-2 is inherently designed with no modifications to the kernel, making it easily portable to other operating systems, including iOS [26] platform.

7 Evaluation

7.1 Experimental Setup

Hardware. We evaluate on two OnePlus [44] smartphones representing different performance tiers: the high-end OnePlus 12 with flag-ship Snapdragon 8 Gen 3 SoC and the mid-end OnePlus Ace 2 with Snapdragon 8+ Gen 1 SoC. Details are listed in Table 3.

Models: We choose five LLMs of varying architectures and sizes, namely Mistral(SiLU)-7B, sparse Qwen2-7B [59], Bamboo-7B [52], sparse Llama-13B [50], and TurboSparse-Mixtral-47B [52] ("Mixtral-47B" in figures for short).

Baselines: We compare PowerInfer-2 with four state-of-the-art LLM inference frameworks: llama.cpp [21], QNN [46], MLC-LLM [53], and LLMFlash [7]. These frameworks represent different approaches to on-device inference: llama.cpp is the most popular CPU-based framework that supports offloading weights to flash storage (via `mmap`), and serves as the backend for many other frameworks like Ollama [3]; QNN is Qualcomm's commercial inference engine with sophisticated and proprietary NPU optimizations; and MLC-LLM leverages mobile GPUs for acceleration. LLMFlash is designed for high-end PC context and not open-sourced. Therefore we implemented LLMFlash's key optimizations in llama.cpp based on the paper's description, including sparsity prediction, row-column bundling, and neuron data caching.

Workloads: We evaluate using representative real-world LLM tasks including multi-turn dialogue [54], code generation [14], math problem solving [18], and role play [55]. These tasks are top representatives of real-world LLM tasks on the HuggingFace platform. Our evaluation covers prefill performance with 128 and 512 token prompts, decoding performance with up to 64 token prompts and 1,024 token outputs, as well as Best-of-N (BoN) generation speed. All experiments are averaged over 10 runs to account for variance.

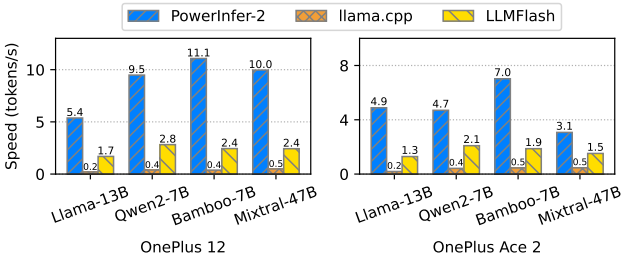


Figure 7: Decoding speeds of PowerInfer-2, llama.cpp and LLMFlash. The Y axis is the generation speed (tokens/s). 50% model weights of FFN blocks are offloaded to flash storage for all models except TurboSparse-Mixtral-47B on OnePlus Ace 2, which requires offloading at least 75% of FFN weights.

7.2 Offloading-Based Performance

In this section, we evaluate PowerInfer-2's decoding and prefill performance across different models under memory-constrained conditions.

7.2.1 Decoding Performance

We limit FFN weight placement in DRAM to 50% across all models, except for TurboSparse-Mixtral-47B on OnePlus Ace 2 which requires 75% offloading due to memory constraints. As shown in Fig.7, PowerInfer-2 demonstrates significant speedups across devices: on OnePlus 12, it achieves average speedups of 3.84× (up to 4.63×) over LLMFlash and 24.6× (up to 27.8×) over llama.cpp. Similarly, on OnePlus Ace 2, PowerInfer-2 delivers average speedups of 2.93× and 14.1× compared to LLMFlash and llama.cpp respectively.

Table 4: Time proportion comparison between Compute and I/O for Bamboo-7B.

System	Compute	I/O
PowerInfer-2	86.3%	13.7%
LLMFlash	23.3%	76.7%

When model size exceeds available DRAM, weight parameters must be dynamically swapped between flash storage and memory, introducing substantial I/O overhead. Although LLMFlash's neuron cache mechanism achieves a 5.35× speedup compared to llama.cpp's direct `mmap` approach, approximately 10% of activated neurons still require loading from flash storage, forcing computation to stall while waiting for random I/O completion.

PowerInfer-2 effectively addresses these challenges through two key innovations: (1) an efficient neuron-cluster-level pipeline that overlaps I/O operations with computation, and (2) flexible bundle loading that maximizes I/O throughput. These techniques effectively eliminate the I/O bottleneck, delivering superior performance across all evaluated models. As demonstrated in Table 4, our analysis of the critical path breakdown reveals that while LLMFlash suffers from significant I/O overhead (76.7%) due to frequent flash storage access with only 23.3% spent on actual computation, PowerInfer-2 dramatically reduces I/O overhead to just 13.7%, substantially improving overall system efficiency.

The acceleration ratio of PowerInfer-2 varies across models due to differences in activated parameters. For instance, while TurboSparse-Mixtral-47B has 47B parameters in total, its mixture-of-expert architecture and high sparsity result in only about 3B activated parameters per token, similar to Bamboo-7B. This explains their comparable performance. At 9.96 tokens/s, TurboSparse-Mixtral-47B still has room for improvement through enlarged neuron cache, as discussed in §7.2.3. In contrast, Llama-13B's lower sparsity leads to nearly 2× more activated parameters than Bamboo-7B, resulting in 2× slower performance.

7.2.2 Prefill Performance

In this section, we evaluate PowerInfer-2's prefill performance with 128 and 512 token prompts. Fig.8 shows the prompt processing speeds across different systems. With 512-token prompts, PowerInfer-2 achieves significant speedups: 48.97× over LLMFlash, 44.23× over llama.cpp, and 1.99× over QNN on OnePlus 12, while showing 29.98× and 16.67× improvements on OnePlus Ace 2. The results are similar for 128-token prompts, which reach up to 22.47× speedup over LLMFlash.

These performance gains stem from PowerInfer-2's NPU-centric prefill and flexible neuron loading mechanism that preloads transformer layers using sequential I/O. The NPU demonstrates superior performance for large batch sizes compared to CPU and GPU. Additionally, PowerInfer-2 effi-

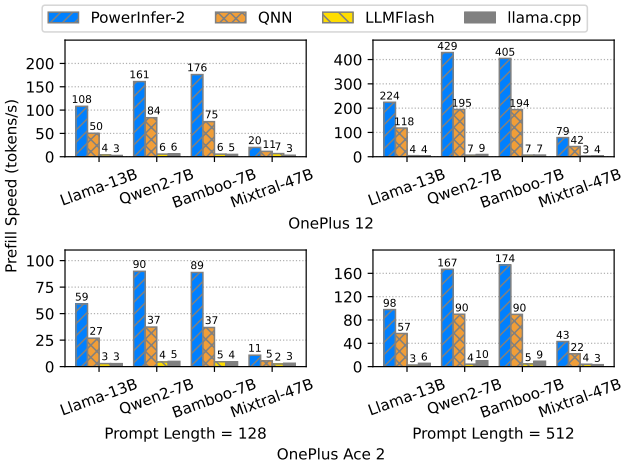


Figure 8: Prefill speeds of PowerInfer-2, QNN, llama.cpp and LLMFlash in offloading scenarios at prompt lengths of 128 and 512 tokens. The X axis is the model. The Y axis is the prompt processing speed (tokens/s). The placement of weights in FFN blocks is the same as Fig.7.

ciently utilizes sequential I/O during prefill. As batch sizes increase, neuron activation probability approaches 99.99% in TurboSparse-Mixtral-47B with 128 batch size. Therefore, PowerInfer-2 sequentially loads entire transformer layers using sequential I/O, which is 3× faster than random I/O. Furthermore, weight loading is efficiently overlapped with computation through prefetching, where weights for the next layer are loaded while computing the current layer. As illustrated in Fig.9, I/O operations are completely overlapped with computation time through our pipelining strategy, effectively hiding I/O latency.

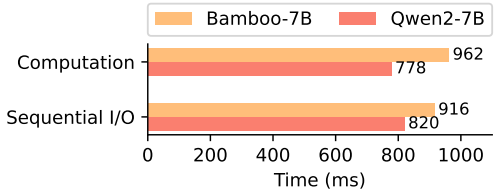


Figure 9: Computation and sequential I/O time in a layer with prompt lengths of 512 tokens at the prefill stage of Bamboo-7B and Qwen2-7B on OnePlus 12. The X axis is time used in milliseconds.

7.2.3 Performance with Various Memory Capacities

In real-world scenarios, smartphones run multiple applications simultaneously [1], leading to varying available memory for LLM inference. We evaluate PowerInfer-2's adaptability across memory capacities from 7GB to 19GB. Fig.10 shows the decoding speeds under different memory configurations for TurboSparse-Mixtral-47B.

With only 7GB available memory, PowerInfer-2 achieves 2.13 tokens/s. The memory is allocated as follows: 1GB for non-FFN layer weights[3], 2.6GB for predictor weights,

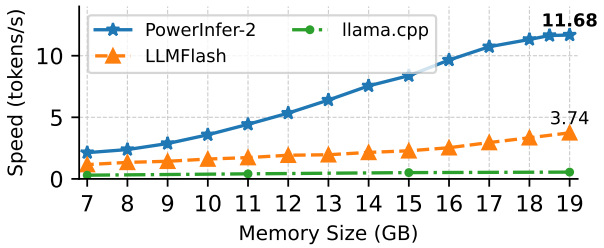


Figure 10: Decoding speeds on various memory configurations with TurboSparse-Mixtral-47B on OnePlus 12.

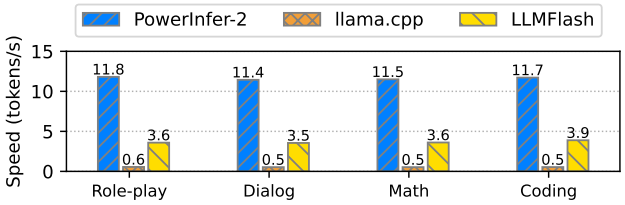


Figure 11: Decoding performance on different downstream tasks of TurboSparse-Mixtral-47B on OnePlus 12. All available memory is used during decoding.

2.7GB for FFN weights' quantization scales, and 300MB for runtime memory, totaling 6.6GB. The remaining 400MB is used for neuron cache, which only stores 1.8% of FFN weights. This small cache size means nearly all neurons must be fetched from flash storage for each token. Despite this limitation, PowerInfer-2 still outperforms LLMFlash by 1.84× through efficient neuron loading and neuron-cluster pipeline. PowerInfer-2's decoding speed scales linearly with memory size up to 19GB, as I/O operations reduce proportionally with increased cache size. At maximum memory utilization (19GB), PowerInfer-2 reaches 11.68 tokens/s, which is 3.12× faster than LLMFlash and 21.2× faster than llama.cpp.

7.2.4 Decoding Consistency Analysis

To evaluate PowerInfer-2's performance consistency, we analyze decoding speeds across different tasks and examine token-level latency distribution. For task-level analysis, we measure decoding speeds across four representative tasks: role-play, multi-turn dialogue, math problem solving, and code generation. As shown in Fig.11, PowerInfer-2 maintains consistent performance with at least 11.4 tokens/s across all tasks, demonstrating robust handling of diverse workloads. Minor speed variations occur due to task-dependent differences in model activation sparsity.

To analyze token-level latency distribution, we measured the average, 50th percentile (P50), 90th percentile (P90), and 99th percentile (P99) decoding speeds across 1,024 tokens for both TurboSparse-Mixtral-47B and Bamboo-7B models with 50% of their FFN weights in DRAM. TurboSparse-Mixtral-47B shows notable performance variations: the slowest 10% of tokens are generated 16.5% below average speed, while

[3]Including token embeddings, self-attention layers, KV cache, and the final language model head

Table 5: Decoding latencies of PowerInfer-2 in milliseconds when offloading 50% of FFN weights.

	TurboSparse-Mixtral-47B	Bamboo-7B
Mean	99.76	90.32
P50	97.42	86.88
P90	116.16	115.02
P99	140.56	162.02

Table 6: Generation speed (tokens/s) comparison for SiLU and ReLU-based LLMs when Offloading 50% FFN weights.

Model	PowerInfer-2	LLMFlash	Speedup
Mistral(SiLU)-7B	5.3	2.18	2.4×
Bamboo-7B	11.1	2.4	4.6×

P99 latency is 40.9% slower than the mean. These variations arise from differing activation patterns between consecutive tokens. When tokens share activation patterns, they benefit from cached neurons, avoiding costly flash storage access. Our analysis reveals that while TurboSparse-Mixtral-47B maintains a low 3.5% average cache miss rate, the P99 miss rate reaches 18.9%. This significant variance in cache efficiency directly impacts token generation latency through increased neuron swapping between cache and flash storage.

7.2.5 SiLU-based LLM Performance

While PowerInfer-2 achieves its remarkable acceleration on ReLU-based models due to their naturally high activation sparsity, it can also effectively accelerate SiLU-based architectures, since recent studies (CATS [31] and CHESS [24]) have shown that SiLU-based LLMs also exhibit approximately 50% activation sparsity. As shown in Table 6, PowerInfer-2 achieves a 2.4× speedup on SiLU-based LLMs. Though more modest than the gains seen with ReLU-based models, this improvement validates the versatility of PowerInfer-2's sparse computation mechanisms across different activation functions. The relatively lower speedup can be attributed to the fundamental characteristics of SiLU-based architectures, where a higher proportion of neurons remain active during inference, creating a performance bottleneck in neuron loading operations.

7.3 In-Memory Performance

This section evaluates Bamboo-7B's performance under sufficient memory conditions, comparing against llama.cpp (CPU), MLC-LLM (GPU) and QNN (NPU) on smartphones, as shown in Fig.12.

For the prefill phase, PowerInfer-2 fully leverages NPU acceleration, achieving comparable performance to QNN with both systems exceeding 700 tokens/s, which is a 32× speedup over other frameworks. PowerInfer-2's NPU-centric approach in prefill stage ensures maximum hardware utilization, though it is slightly slower than QNN due to mixed precision quantization used by PowerInfer-2, a deliberate trade-off for accuracy preservation (will be discussed in §7.6). With 50% FFN weight offloading, PowerInfer-2 maintains 404.6 tokens/s throughput, outperforming QNN through efficient pipelining that overlaps weight loading operations. The performance gap compared to the in-memory setup is primarily due to the additional I/O overhead from flash storage access.

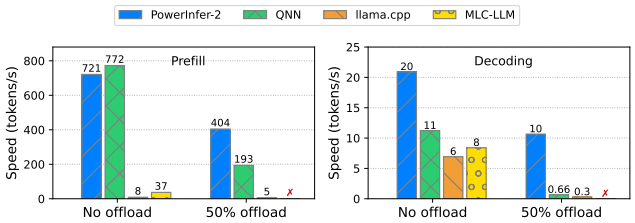


Figure 12: Decoding speeds of PowerInfer-2, llama.cpp, and MLC-LLM on Bamboo-7B with different offloading setups. "50% offload" means 50% model weights of FFN blocks are offloaded to flash storage. "No offload" means all model parameters are resident in memory. A red label of "✗" indicates an execution failure due to the lack of weight offloading support.

In the decoding phase, PowerInfer-2 demonstrates 2.24×, 2.48× and 1.86× speedups over llama.cpp, MLC-LLM and QNN respectively. This superior performance comes from: 1) concurrent CPU-NPU computation improving memory bandwidth utilization from 43.9GB/s to 59.6GB/s, and 2) leveraging the model's activation sparsity to reduce FFN computations.

When offloading 50% of FFN weights, PowerInfer-2 reduces memory usage by 1.5GB (40%) while maintaining performance comparable to llama.cpp and MLC-LLM. This demonstrates PowerInfer-2's ability to optimize memory efficiency without compromising user experience, even for models that already fit in memory.

7.4 Best-of-N Sampling

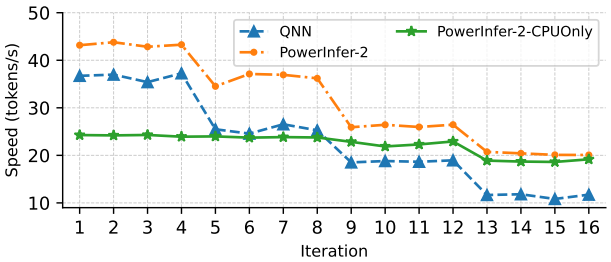


Figure 13: Decoding speed curves of PowerInfer-2, QNN, and PowerInfer-2-CPUOnly on Bamboo-7B under the setting of Best-of-N (N=4). The batch size decreases by one every four iterations. X axis refers to iteration. Each iteration represents the generation of one token across N candidates in parallel. Y axis refers to the generation speed (tokens/s). All model parameters are resident in memory.

Best-of-N (BoN) sampling, where N different response sequences are generated for one prompt and the best one is selected, is widely utilized in LLM applications. We evaluated Best-of-4 sampling on Bamboo-7B using three con-

figurations: PowerInfer-2, QNN, and PowerInfer-2-CPUOnly (PowerInfer-2 with CPU-only decoding). As shown in Fig.13, in the initial phase with all four samples generating simultaneously, PowerInfer-2 achieves 1.84× and 1.28× speedups over QNN and PowerInfer-2-CPUOnly respectively. As samples complete and N decreases, PowerInfer-2 maintains a 1.42× performance advantage through dynamic CPU-NPU task dispatching based on sparsity patterns. When N reduces to 1, increased sparsity causes QNN's performance to drop below PowerInfer-2-CPUOnly. However, PowerInfer-2 still achieves 1.1× and 1.77× speedups over PowerInfer-2-CPUOnly and QNN respectively, benefiting from its hybrid CPU/NPU computation strategy.

7.5 Performance Breakdown

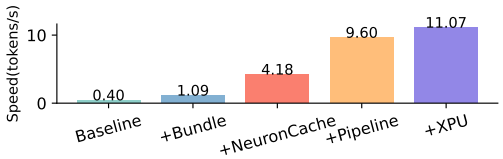


Figure 14: Performance breakdown for decoding phase of the Bamboo-7B model on OnePlus 12, with 50% FFN weights offloaded to flash storage.

To quantify each optimization's contribution, we conducted an ablation study by incrementally enabling optimizations. Starting from a baseline configuration, we progressively added Bundle, Neuron Cache, Neuron-Cluster-Level Pipeline, and XPU optimizations. Fig.14 shows the performance impact of each addition.

The baseline system utilizes CPU without any optimizations and achieves 0.4 tokens/s without optimizations. Adding Bundle optimization improves this to 1.1 tokens/s by increasing I/O efficiency through bundled memory access operations. Neuron Cache further brings a 2.82× speedup (4.181 tokens/s) by achieving a 95% cache hit rate, significantly reducing Flash I/O bandwidth pressure. Pipeline optimization adds another 1.29× improvement (9.60 tokens/s) through overlapped computation and memory access. Finally, XPU optimization provides an additional 15% gain (11.07 tokens/s) by utilizing both CPU and XPU for concurrent memory access.

7.6 Accuracy Evaluation

We evaluate inference accuracy using OpenCompass [19], a widely-adopted benchmark suite, focusing on INT4 quantization which is the de facto standard in real-world mobile deployments [38, 56]. As shown in Table 7, PowerInfer-2 maintains comparable accuracy to llama.cpp across different models and tasks, while significantly outperforming QNN. The accuracy gap between frameworks stems from their different quantization approaches. QNN uses per-channel quantization (one scale per weight matrix row), which poorly handles weights with outlier values. llama.cpp achieves better accuracy by quantizing weights in groups of 32. Since NPUs don't support group-wise quantization, PowerInfer-2 adopts a hybrid quantization approach that handles outlier weights in INT8 precision while applying INT4 per-channel quantization to remaining weights, effectively preserving accuracy while maintaining fast inference speed similar to QNN.

Table 7: Comparison of LLM accuracy between different quantization methods on Bamboo 7B and Qwen2 7B models. The evaluation metrics include Arc Challenge [16], Arc Easy, MMLU [25], and GSM8K [17] benchmarks.

Model	Framework	Average	Arc-Challenge	Arc-Easy	MMLU	GSM8K
Qwen2 7B	llama.cpp	79.25	82.37	87.46	69.72	77.43
	QNN	56.93	65.76	80.25	59.45	22.26
	PowerInfer-2	78.38	81.36	88.18	68.73	75.24
Bamboo 7B	llama.cpp	70.12	73.90	84.83	61.24	60.50
	QNN	63.26	69.15	78.31	57.00	48.59
	PowerInfer-2	68.35	71.86	82.01	61.48	58.05

7.7 Energy Consumption

Table 8: Energy consumption comparison between different frameworks.

Framework	PowerInfer-2	QNN	llama.cpp
Peak Power (W)	5.095	5.133	4.065
Energy (J/token)	0.257	0.373	0.672

We evaluate the energy efficiency of PowerInfer-2 through evaluation of end-to-end energy consumption during inference. Our primary metric is Joules per token (J/token), which quantifies the average energy required to generate each output token. To ensure representative results, we randomly sampled 100 prompts from the lmsys/lmsys-chat-1m [63] dataset, which contains real-world user interactions collected from chatbot deployments. The mean energy consumption results are shown in Table 8. PowerInfer-2 demonstrates superior energy efficiency, achieving 0.257 J/token, which represents a 31.1% and 61.8% reduction in energy consumption compared to QNN and llama.cpp, respectively. This enhanced efficiency is attributed to PowerInfer-2's ability to maintain similar peak power while achieving faster computation speeds, resulting in lower overall energy consumption.

8 Related Work

Resource-Efficient DNN/LLM Inference. The deployment of DNNs and LLMs on resource-constrained devices has gained significant traction recently [58]. While BAND [28] demonstrates the feasibility of running multiple compact DNNs on a single XPU, its methodology does not address the memory-flash hierarchy and cannot scale to accommodate LLMs due to their substantial computational demands. MLC-LLM [53] represents a significant advancement in enabling GPU-accelerated LLM deployment on mobile devices. Nevertheless, it is constrained by the requirement that models must fit entirely in memory. Although mllm-NPU [57] leverages on-device NPU, it lacks CPU-NPU co-processing and requires models to fit in memory, limiting its use for larger models. While STI [23] introduces pipeline mechanisms for I/O-compute overlapping, its capability is limited to

BERT-scale models (0.33B parameters) on embedded devices, making it insufficient for modern LLMs that typically exceed 1B parameters. Various model compression techniques have emerged, including network pruning [27, 40], knowledge distillation [29], and quantization [13, 36], all aimed at reducing model memory footprints. These approaches complement PowerInfer-2 and can be integrated with it to further enhance the efficiency of LLM deployment on mobile devices.

Speculative Decoding. Speculative decoding accelerates inference by leveraging a smaller model to predict tokens in parallel [12, 20, 33]. The technique employs a smaller model (e.g., 1B) to generate candidate tokens, which are then validated in batch by a larger model (e.g., 13B). SpecInfer [41] demonstrates that this approach effectively reduces decoding iterations. While speculative decoding and sparse activation are orthogonal optimization strategies, their integration in memory-constrained XPU environments remains an open research challenge.

9 Conclusion

This paper presents PowerInfer-2, a mobile-centric LLM inference framework that effectively runs LLMs on smartphones with limited memory. Evaluation demonstrates that PowerInfer-2 achieves up to 27.8× speedup over existing solutions while supporting models as large as 47B with minimal accuracy loss.

References

[1] Multitasking the Android Way. https://android-developers.googleblog.com/2010/04/multitasking-android-way.html, 2010.

[2] Huawei's LLM on Mate 70 Series. https://www.huaweicentral.com/huawei-mate-70-series-launched-with-new-camera-design-5700mah-battery-generative-ai-and-more/, 2024.

[3] Ollama: Get up and running with large language models locally. https://github.com/ollama/ollama, 2024.

[4] OPPO Announces Commitment to Making AI Phones Accessible to Everyone. https://www.oppo.com/en/newsroom/press/oppo-make-ai-phones-accessible/, 2024.

[5] Samsung's LLM on Galaxy S24. https://www.samsung.com/global/galaxy/s24/, 2024.

[6] Abien Fred Agarap. Deep Learning using Rectified Linear Units (ReLU), 2019.

[7] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. LLM in a Flash: Efficient Large Language Model Inference with Limited Memory, 2024.

[8] Android. Android 15. https://www.android.com/, 2024.

[9] Anthropic. https://www.anthropic.com/news/claude-3-family, 2024.

[10] Apple. Apple Intelligence. https://www.apple.com/apple-intelligence/, 2024.

[11] ARM. Arm Big.LITTLE. https://www.arm.com/zh-TW/technologies/big-little, 2024.

[12] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. *arXiv preprint arXiv: 2401.10774*, 2024.

[13] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher M De Sa. QuIP: 2-bit quantization of large language models with guarantees. *Advances in Neural Information Processing Systems*, 36, 2024.

[14] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating Large Language Models Trained on Code, 2021.

[15] Yidong Chen, Chen Zhang, Rongchao Dong, Haoyuan Zhang, Yonghua Zhang, Zhonghua Lu, and Jidong Zhai. MixQ: Taming Dynamic Outliers in Mixed-Precision Quantization by Online Prediction. In *2024 SC24: International Conference for High Performance Computing, Networking, Storage and Analysis SC*, pages 1161–1175. IEEE Computer Society, 2024.

[16] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

[17] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems, 2021. *URL https://arxiv.org/abs/2110.14168*, 2021.

[18] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*, 2021.

[19] OpenCompass Contributors. OpenCompass: A Universal Evaluation Platform for Foundation Models. https://github.com/open-compass/opencompass, 2023.

[20] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the Sequential Dependency of LLM Inference Using Lookahead Decoding, 2024.

[21] Georgi Gerganov. ggerganov/llama.cpp: Port of Facebook's LLaMA model in C/C++. https://github.com/ggerganov/llama.cpp, 2024.

[22] Google. Get started with Gemini Nano on Android (on-device). https://ai.google.dev/gemini-api/docs/get-started/android_aicore, 2024.

[23] Liwei Guo, Wonkyo Choe, and Felix Xiaozhu Lin. STI: Turbocharge NLP Inference at the Edge via Elastic Pipelining. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS 2023, page 791–803, New York, NY, USA, 2023. Association for Computing Machinery.

[24] Junhui He, Shangyu Wu, Weidong Wen, Chun Jason Xue, and Qingan Li. CHESS: Optimizing LLM Inference via Channel-Wise Thresholding and Selective Sparsification. *arXiv preprint arXiv:2409.01366*, 2024.

[25] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

[26] iOS. iOS 17. https://www.apple.com/ios/ios-17/, 2024.

[27] Ajay Jaiswal, Shiwei Liu, Tianlong Chen, Zhangyang Wang, et al. The emergence of essential sparsity in large pre-trained models: The weights that matter. *Advances in Neural Information Processing Systems*, 36, 2024.

[28] Joo Seong Jeong, Jingyu Lee, Donghyun Kim, Changmin Jeon, Changjin Jeong, Youngki Lee, and Byung-Gon Chun. Band: coordinated multi-DNN inference on heterogeneous mobile processors. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services*, MobiSys '22, page 235–247, New York, NY, USA, 2022. Association for Computing Machinery.

[29] Jaehun Jung, Peter West, Liwei Jiang, Faeze Brahman, Ximing Lu, Jillian Fisher, Taylor Sorensen, and Yejin Choi. Impossible distillation: from low-quality model to high-quality dataset & model for summarization and paraphrasing. *arXiv preprint arXiv:2305.16635*, 2023.

[30] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling Laws for Neural Language Models, 2020.

[31] Je-Yong Lee, Donghyun Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. CATS: Contextually-Aware Thresholding for Sparsity in Large Language Models. *arXiv preprint arXiv:2404.08763*, 2024.

[32] Luchang Li, Sheng Qian, Jie Lu, Lunxi Yuan, Rui Wang, and Qin Xie. Transformer-Lite: High-efficiency Deployment of Large Language Models on Mobile Phone GPUs, 2024.

[33] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. *arXiv preprint arXiv:2401.15077*, 2024.

[34] Zonglin Li, Chong You, Srinadh Bhojanapalli, Daliang Li, Ankit Singh Rawat, Sashank J. Reddi, Ke Ye, Felix Chern, Felix Yu, Ruiqi Guo, and Sanjiv Kumar. The Lazy Neuron Phenomenon: On Emergence of Activation Sparsity in Transformers, 2023.

[35] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

[36] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. *Proceedings of Machine Learning and Systems*, 6:87–100, 2024.

[37] Zichang Liu, Jue Wang, Tri Dao, Tianyi Zhou, Binhang Yuan, Zhao Song, Anshumali Shrivastava, Ce Zhang, Yuandong Tian, Christopher Re, and Beidi Chen. Deja Vu: Contextual Sparsity for Efficient LLMs at Inference Time, 2023.

[38] Xudong Lu, Yinghao Chen, Cheng Chen, Hui Tan, Boheng Chen, Yina Xie, Rui Hu, Guanxin Tan, Renshou Wu, Yan Hu, et al. BlueLM-V-3B: Algorithm and System Co-Design for Multimodal Large Language Models on Mobile Devices. *arXiv preprint arXiv:2411.10640*, 2024.

[39] Yuqi Luo, Chenyang Song, Xu Han, Yingfa Chen, Chaojun Xiao, Zhiyuan Liu, and Maosong Sun. Sparsing Law: Towards Large Language Models with Greater Activation Sparsity, 2024.

[40] Xinyin Ma, Gongfan Fang, and Xinchao Wang. LLM-Pruner: On the structural pruning of large language models. *Advances in neural information processing systems*, 36:21702–21720, 2023.

[41] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. SpecInfer: Accelerating Generative Large Language Model Serving with Speculative Inference and Token Tree Verification, 2023.

[42] NVIDIA. https://www.nvidia.com/en-us/data-center/h100/, 2024.

[43] NVIDIA. NVIDIA GB200. https://www.nvidia.com/en-us/data-center/gb200-nvl72, 2024.

[44] OnePlus. OnePlus Global. https://www.oneplus.com/global, 2024.

[45] OpenAI. https://openai.com/gpt-4, 2023.

[46] Qualcomm. Qualcomm Neural Processing SDK. https://developer.qualcomm.com/software/qualcomm-neural-processing-sdk, 2023.

[47] Konstantinos I Roumeliotis, Nikolaos D Tselikas, and Dimitrios K Nasiopoulos. Llama 2: Early Adopters' Utilization of Meta's New Open-Source Pretrained Model. 2023.

[48] Noam Shazeer. GLU Variants Improve Transformer, 2020.

[49] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

[50] Chenyang Song, Xu Han, Zhengyan Zhang, Shengding Hu, Xiyu Shi, Kuai Li, Chen Chen, Zhiyuan Liu, Guangli Li, Tao Yang, and Maosong Sun. ProSparse: Introducing and Enhancing Intrinsic Activation Sparsity within Large Language Models. *arXiv preprint arXiv:2402.13516*, 2024.

[51] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, SOSP '24, page 590–606, New York, NY, USA, 2024. Association for Computing Machinery.

[52] Yixin Song, Haotong Xie, Zhengyan Zhang, Bo Wen, Li Ma, Zeyu Mi, and Haibo Chen. Turbo Sparse: Achieving LLM SOTA Performance with Minimal Activated Parameters. *arXiv preprint arXiv:2406.05955*, 2024.

[53] MLC team. MLC-LLM. https://github.com/mlc-ai/mlc-llm, 2024.

[54] Lewis Tunstall, Edward Beeching, Nathan Lambert, Nazneen Rajani, Kashif Rasul, Younes Belkada, Shengyi Huang, Leandro von Werra, Clémentine Fourrier, Nathan Habib, Nathan Sarrazin, Omar Sanseviero, Alexander M. Rush, and Thomas Wolf. Zephyr: Direct Distillation of LM Alignment, 2023.

[55] Zekun Moore Wang, Zhongyuan Peng, Haoran Que, Jiaheng Liu, Wangchunshu Zhou, Yuhan Wu, Hongcheng Guo, Ruitong Gan, Zehao Ni, Man Zhang, Zhaoxiang Zhang, Wanli Ouyang, Ke Xu, Wenhu Chen, Jie Fu, and Junran Peng. RoleLLM: Benchmarking, Eliciting, and Enhancing Role-Playing Abilities of Large Language Models. *arXiv preprint arXiv:2310.00746*, 2023.

[56] Jie Xiao, Qianyi Huang, Xu Chen, and Chen Tian. Large Language Model Performance Benchmarking on Mobile Platforms: A Thorough Evaluation. *arXiv preprint arXiv:2410.03613*, 2024.

[57] Daliang Xu, Hao Zhang, Liming Yang, Ruiqi Liu, Gang Huang, Mengwei Xu, and Xuanzhe Liu. Empowering 1000 tokens/second on-device llm prefilling with mllm-npu. *arXiv preprint arXiv:2407.05858*, 2024.

[58] Mengwei Xu, Wangsong Yin, Dongqi Cai, Rongjie Yi, Daliang Xu, Qipeng Wang, Bingyang Wu, Yihao Zhao, Chen Yang, Shihe Wang, et al. A survey of resource-efficient LLM and multimodal foundation models. *arXiv preprint arXiv:2401.08092*, 2024.

[59] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.

[60] Di Zhang, Xiaoshui Huang, Dongzhan Zhou, Yuqiang Li, and Wanli Ouyang. Accessing gpt-4 level mathematical olympiad solutions via monte carlo tree self-refine with llama-3 8b. *arXiv preprint arXiv:2406.07394*, 2024.

[61] Zhengyan Zhang, Yankai Lin, Zhiyuan Liu, Peng Li, Maosong Sun, and Jie Zhou. MoEfication: Transformer Feed-forward Layers are Mixtures of Experts. In *Findings of ACL 2022*, 2022.

[62] Zhengyan Zhang, Yixin Song, Guanghui Yu, Xu Han, Yankai Lin, Chaojun Xiao, Chenyang Song, Zhiyuan Liu, Zeyu Mi, and Maosong Sun. ReLU^2 Wins: Discovering Efficient Activation Functions for Sparse LLMs, 2024.

[63] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric. P Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. LMSYS-Chat-1M: A Large-Scale Real-World LLM Conversation Dataset, 2023.

[64] Zihan Zhou, Chong Li, Xinyi Chen, Shuo Wang, Yu Chao, Zhili Li, Haoyu Wang, Rongqiao An, Qi Shi, Zhixing Tan, Xu Han, Xiaodong Shi, Zhiyuan Liu, and Maosong Sun. LLM×MapReduce: Simplified Long-Sequence Processing using Large Language Models, 2024.